Extension of UAV Optimal Controller to Multi-Modal Control

Ethan Torres
ethanjt2

February 15, 2026

Contents

1 Introduction

The paper I've chosen to analyze, *Real-Time Stochastic Optimal Control for Multi-Agent Quadrotor systems* [1], explores innovative methodologies for controlling multi-agent quadrotor systems through real-time stochastic optimal control. The proposed approach integrates advanced control techniques with hierarchical planning to address challenges associated with uncertainty, scalability, and coordination in dynamic environments. By leveraging theoretical foundations and practical implementations, the study aims to demonstrate the efficacy of these methods in both simulated and real-world settings, contributing to the broader field of autonomous UAV research. My analysis is two-fold: on one hand I work through and explain the proofs presented in the paper that explain the theoretical underpinning of the stochastic optimal control problem that is presented. On the other hand, I present a clear potential continuation of this research as it relates to my own research in high dimensional multi-agent reinforcement learning with deep neural networks. While I do not explicitly explore these methods via coding and testing, I do show the natural mathematical extension of this stochastic control problem and its formulation to Deep MARL (multi-agent reinforcement learning). Moreover, I provide a clear road map for anyone who wants to actively pursue research in that area centered around the developments in UAV control presented in this problem. Finally, this paper allowed me to strengthen the mathematics behind my own research in Deep MARL where I use a similar problem with similar dynamics and a similar stochastic control problem setup, but with different external and internal forces affecting the system.

2 Retrieval of State Vector for UAV

One of the most important aspects of both solving the problem in [**?**] and then trying to extend it to Deep MARL is the construction of the environment and the physics that determine its state vector. This state vector is critical in determining updates in the optimal control problem and how it is processed is itself quite a complex problem due to the complexity of the dynamics. The quadrotor system exhibits non-linear, multi-dimensional dynamics characterized by yaw, pitch, and roll, which are interpreted and managed by PID controllers. The flight control system (FCS) provides target control to the UAV in the form of a velocity update, $\mathbf{v} = [v_E, v_N]^\top$, which is subsequently passed along with the height $\hat{p}_U$ to a Velocity-Height controller [1]. This controller utilizes the current state estimate of the quadrotor system, represented as $\mathbf{y} = [p_E, p_N, p_U, \phi, \theta, \varphi, u, v, w, p, q, r]^\top$, where $[p_E, p_N, p_U]$ and (ϕ, θ, φ) denote navigation-frame position and orientation, and (u, v, w) and (p, q, r) are the body-frame and angular velocities, respectively.

The system relies on four independent PID controllers to regulate roll ($\hat{\phi}$), pitch ($\hat{\theta}$), throttle ($\hat{\gamma}$), and yaw rate ($\hat{r}$), sending these commands to the FCS as illustrated in Figure 2. As demonstrated later, achieving stable autonomous control with this flight controller aligns naturally with multi-agent reinforcement learning (MARL). By developing an optimal policy, actions can be sent to the FCS, state updates received from the PID controllers, and iterative adjustments made using running cost-to-go functions and corresponding reward updates for Q-learning (detailed in the MARL section).

It should be noted that the MARL approach presented is based on my prior research on a related problem, which is why references to MARL-specific sources are omitted.

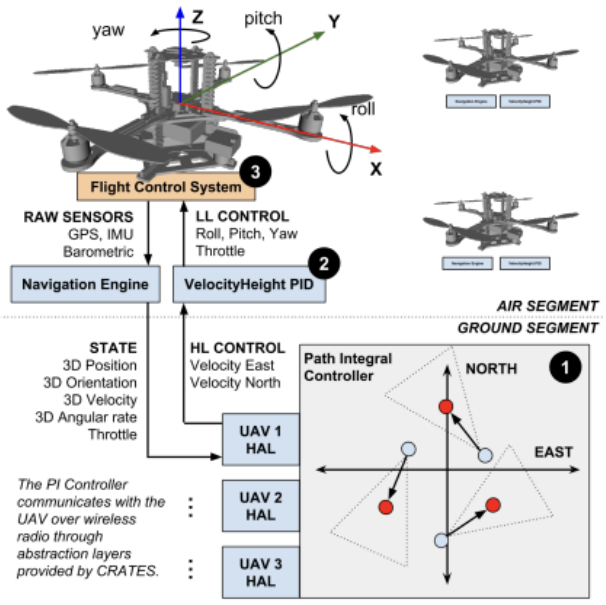

Figure 1: **Control hierarchy**: The path-integral controller (1) calculates target velocities/heights for each quadrotor. These are converted to roll, pitch, throttle and yaw rates by a platform-specific Velocity Height PID controller (2). This control is in turn passed to the platform's flight control system (3), and converted to relative motor speed changes.

Figure 1: PI Controller

3 Path Integral Control

3.1 Path Integral Formulation

The main problem that the paper attempts to solve is a Path-Integral stochastic optimal control problem. We are presented with a very canonical formulation of a continuous stochastic optimal control problem with system dynamics:

$$\begin{aligned} d\mathbf{x} &= \mathbf{f}(x,t)dt + \mathbf{G}(\mathbf{x},t)(\mathbf{u}(t)dt + d\boldsymbol{\xi}) \\ &= \mathbf{f}(x,t)dt + \mathbf{G}(\mathbf{x},t)\mathbf{u}(t)dt + \mathbf{G}(\mathbf{x},t)d\boldsymbol{\xi} \end{aligned}$$

where $\mathbf{x} \in \mathbb{R}^n$ is the state vector, $\mathbf{u}(t) \in \mathbb{R}^n$ is the control input vector, and $\mathbf{w}(t)$ is an m-dimensional Weiner process. It is also defined that $\mathbf{G}(\mathbf{x},t) \in \mathbb{R}^{n \times m}$ and covariance $\sum_u \in \mathbb{R}^{m \times m}$. Let it be clear that the function $\mathbf{f}(\mathbf{x},t)$ is the uncontrolled drift dynamics associated with physical inputs like the Coriolis force, gravity, and centripetal forces. We can then define a realization of the path $\boldsymbol{\tau} = \mathbf{x}_{0:dt:T}$ as is typically done. The authors go on to describe the dynamics subject to the cost-to-go function:

$$S(\boldsymbol{\tau}|\mathbf{x}_0, \mathbf{u}) = r_T(\mathbf{x}_T) + \sum_{t=0:dt:T-dt} \left(r_t(\mathbf{x}_t) + \frac{1}{2}\mathbf{u}_t^T \mathbf{R} \mathbf{u}_t \right)$$

where we have the running cost r_t and the terminal cost r_T w.r.t the control:

$$\mathbf{u}^* = \arg\min_{\mathbf{u}} \mathbb{E}[S(\boldsymbol{\tau}|\mathbf{x}_0, \mathbf{u})]$$

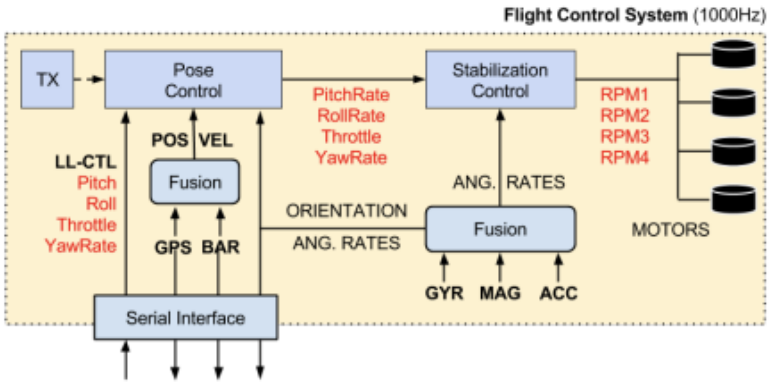

Figure 2: FCS (Flight Control System)

However, it clearly becomes apparent that we can rewrite this formulation of the cost-to-go function subject to the constraint such that we have:

$$S(\mathbf{x}, t) = \min_{\mathbf{u}} \mathbb{E}\Big[\int_t^T \Big(\frac{1}{2} \mathbf{u}^T \mathbf{R} \mathbf{u} + r_t(\mathbf{x}, \tau) d\tau \Big) \Big] + r_T(\mathbf{x}(T))$$

where now we have reformulated the problem as is done in [2]. The point of this is hopefully we can reduce the problem as the authors have done into taking the expectation under the optimal path distribution. We can do this by applying the Feynman-Kac formula and express the cost-to-go function as:

$$S(\mathbf{x}, t) = -\lambda \log \mathbb{E}_{p_0} \left[\exp \left(-\frac{1}{\lambda} \int_t^T r_t(\mathbf{x}(\tau), \tau) \, d\tau - \frac{1}{\lambda} r_T(\mathbf{x}(T)) \right) \right]$$

where we have now defined a control cost parameter λ. Now, we can take this minimum by taking the gradient with respect to the state $\mathbf{x}$. The gradient of S is:

$$\nabla S(\mathbf{x}, t) = -\lambda \nabla \log \mathbb{E}_{p_0} \left[\exp \left(-\frac{1}{\lambda} \int_t^T r_t(\mathbf{x}(\tau), \tau) \, d\tau - \frac{1}{\lambda} r_T(\mathbf{x}(T)) \right) \right]$$

we then exploit the identity $\nabla \log Z = \frac{\nabla Z}{Z}$ and we rewrite this derivative as:

$$\nabla S(\mathbf{x}, t) = -\lambda \frac{\nabla \mathbb{E}_{p_0} \left[\exp \left(-\frac{1}{\lambda} \int_t^T r_t(\mathbf{x}(\tau), \tau) \, d\tau - \frac{1}{\lambda} r_T(\mathbf{x}(T)) \right) \right]}{\mathbb{E}_{p_0} \left[\exp \left(-\frac{1}{\lambda} \int_t^T r_t(\mathbf{x}(\tau), \tau) \, d\tau - \frac{1}{\lambda} r_T(\mathbf{x}(T)) \right) \right]}$$

Here, the numerator involves reweighting the paths where we have exponential weighting:

$$w(\mathbf{x}(\tau)) = e^{\left[\left(-\frac{1}{\lambda} \int_t^T r_t(\mathbf{x}(\tau), \tau) \, d\tau - \frac{1}{\lambda} r_T(\mathbf{x}(T)) \right) \right]}$$

Hence, when we rewrite the dynamics equation as:

$$\mathbf{u}^*(\mathbf{x}, t) = \mathbf{G}^T(\mathbf{x}, t) \nabla S(\mathbf{x}, t)$$

we get a rewritten cost-to-go function as:

$$\nabla S(\mathbf{x}, t) \propto \mathbb{E}_{p^*} [\nabla \log p^*(\mathbf{x}(t))]$$

Finally, we have shown how the authors have achieved the expression of the solution of the optimal control problem as the expectation under the optimal path distribution by using the reweighting of the paths with the proven exponential weighting $w(\mathbf{x}(\tau))$ and rewriting using paths (stochastic process) p:

$$p^*(\boldsymbol{\tau}|\mathbf{x}_0) \propto p(\boldsymbol{\tau}|\mathbf{x}_0, \mathbf{u}) e^{-S(\frac{\boldsymbol{\tau}|\mathbf{x}_0, \mathbf{u}}{\lambda})}$$

$$\langle \mathbf{u}_t^*(\mathbf{x}_0) \rangle = \langle \mathbf{u}_t + \frac{(\xi_{t+dt} - \xi_t)}{dt} \rangle$$

where the first equation we proved by deriving the exponential weighting of the path distributions (we proved that $e^{-S(\frac{\boldsymbol{\tau}|\mathbf{x}_0, \mathbf{u}}{\lambda})} = w(\mathbf{x}(t))$ and that it is true) and the second equation we have proved by taking the gradient in order to minimize and have discretized this derivative in order to find the minimum. It should also be kept in mind that $\langle \cdot \rangle$ is used to represent the expectation operator as is reflected in [1] as this notation is not conventional. In summary, there is a very natural solution to model the dynamics as a path-integral control problem where the authors have utilized techniques in stochastic optimal control theory to minimize a cost-to-go function in hopes of optimally traversing space coordinates to reach some predetermined goal.

3.2 Extension to Multi-Modal Control

We then need to turn to the problem of introducing multiple agents. To that we look at section 4.3 in [2]. Clearly, we want to optimize over all the paths that all the agents can take so we are left with the problem of:

$$\mathbf{u}^* = \arg\min_{\mathbf{u}} \mathbb{E}[S(\boldsymbol{\tau}|\mathbf{x}_0, \mathbf{u})] = \arg\min_{\mathbf{u}} \mathbb{E}_{p^*}[\nabla S(\boldsymbol{\tau}|\mathbf{x}_0, \mathbf{u})]$$

which is ostensibly taking the gradient of the expectation of all the optimal paths possible and then taking the argmin of those paths to return a single path and escape the local minima (this will be even more relevant when one wants to apply MARL (multi-agent Reinforcement Learning) for automatic control of the aircraft where local minima can cause the network or Q function to get trapped and the policy to never converge). Kappen [2] develops quite a nuanced argument that I would encourage the reader to investigate but I will not include for brevity (the Laplace derivation is not necessary). We are then left with an optimal control that becomes a soft-max of the deterministic strategies:

$$u(x,t) = -R^{-1} \sum_{\alpha} w_\alpha \partial_x S(x,t)$$

$$w_\alpha = \frac{e^{-S(x,t)}}{\sum_k e^{-S_k(x,t)}}$$

where here, in the context of [1] we have written the constraint $\sum_u \lambda \mathbf{R}^{-1}$ which forces control and noise to act in the same dimensions in an inverse relation. Clearly, this method allows for a clear algorithm to be written that discretizes each of the paths possible for each of the agents, as well as taking into account multiple agents having multiple local minima by adding in both the weighting term w_α and the force control $\lambda \mathbf{R}^{-1}$. This of course can be applied to the above proof in [2.1] to extend our definition.

4 Application to Multi-UAV Planning

We are then left with the very clear algorithm for a PIController for multi-agent UAV motion planning where we have utilized both the proof in [2.1] and the extension of that proof to multiple agents in

Algorithm 1 PI Control for UAV motion planning

function PICONTROLLER(N,H,dt,$r_t(\cdot)$,$\sum_u$,$\mathbf{u}_{t:dt:t+H}$)
  for $k = 1, ..., N$ **do**
    Sample Paths $\boldsymbol{\tau}_k = \{\mathbf{x}_{t:dt:t+H}\}_k$
  end for
  Compute $S_k = S(\boldsymbol{\tau}|x_0, \mathbf{u})$ (follow S above)
  Store the noise realizations $\{\boldsymbol{\xi}_{t:dt:t+H}\}_k$
  Compute the weights as in [2.2]: $w_k = e^{-S_k/\lambda} / \sum_l e^{-S_l/\lambda}$
  for $s = t : dt : t + H$ **do**
    $\mathbf{u}^*_s = \mathbf{u}_s + \frac{1}{dt} \sum_k w_k (\{\boldsymbol{\xi}_{s+dt}\}_k - \{\boldsymbol{\xi}_s\}_k)$
  end for
  Return: next desired velocity: $v_{t+dt} = v_t + \mathbf{u}^*_t dt$ and $\mathbf{u}^*_{t:dt:t+H}$ for importance sampling at $t + dt$
end function

[2.2]:
What this function does is take a collection of sampled paths $\boldsymbol{\tau}_k$ from the decoupled linear form of the dynamics:

$$dx_i = Ax_i dt + B(u_i dt + d\xi_i)$$
$$A = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}, \quad B = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

where we have $\boldsymbol{x}$, the state vector, which is composed the EN positions and velocities of each agent $i = 1, ..., M$ as $x_i = [p_i, v_i]^\top \in \mathbb{R}^2$. Then the algorithm computes S based on the true dynamics of hte system where the state cost $r_t(\boldsymbol{x}_t)$ can be any arbitrary function of all UAVs states which returns us to the non-linear coupled state equation S above. From there, it calculates noise realizations according to the Wiener process ξ_t incremented by time. Then as shown above in section [2.2], the weights are computed for the multi-modal system because there are multiple agents at play and this must be mathematically and systematically accounted for. Finally, the optimal control $\boldsymbol{u}^*_t$ is calculated by following the above dynamics with the interpreted state vector by the controllers, weighting the noise realizations and then iteratively updating $\boldsymbol{u}_{t:dt:t+H}$ with the importance control sequence. This process ensures that the environment's dynamics are accurately interpreted, enabling the controllers to respond appropriately. Such a framework is well-suited for automation using AI, particularly through Deep Multi-Agent Reinforcement Learning (DMARL). In DMARL, state vectors are processed, dynamics are computed (as described by the iterative algorithm above), and an RL agent learns the optimal actions required to achieve a predefined goal. These goals could encompass a variety of scenarios, including all those outlined in the referenced paper. However, for the sake of brevity, this discussion will focus on the "holding problem," where agents are tasked with reaching specific positions in space and maintaining a stable state at those positions. The implementation will explore the construction of the agent required to accomplish this task effectively.

5 Extensions to MARL

There is an obvious extension that becomes apparent to MARL (multi-agent reinforcement learning) where we can autonomously control multiple UAVs to do a multitude of the scenarios that were presented. The example that we will pursue here is a holding pattern. We first show the well known Bellman ODE for Q-Learning:

$$Q^*(s, a) = R(s, a) + \gamma \min_{a' \in A} Q(\Phi_a(x), a')$$

where we have some initial state: $z_0 = z$ which coincides with an initial policy: $\Phi_a^{(0)}(z_0) := z$ and an optimal policy: $\arg\max_i \Phi_i(z) = \Phi^*(z)$. The idea, is that we can develop a reward function that coincides with the cost-to-go running cost function for holding patterns. This is clearly derived in [1]:

$$r_{HP} = \sum_{i=1}^{M} \exp(v_i - v_{max}) + \exp(v_{min} - v_i) + \exp(||p_i - d||_2) + \sum_{j>i}^{M} \frac{C_{hit}}{||p_i - p_j||_2}$$

where v_{max} and v_{min} are the maximum and minimum velocities respectively, d is the penalty for deviation from the origin, and C_{hit} is the penalty associated with collision risk of two agents. $||\cdot||_2$ is the $\ell - 2$ norm. We can thus create our reward to give us the following modification of the Bellman ODE:

$$Q^*(s,a) = \sum_{i=1}^{M} \exp(v_i - v_{max}) + \exp(v_{min} - v_i) + \exp(||p_i - d||_2) + \sum_{j>i}^{M} \frac{C_{hit}}{||p_i - p_j||_2} + \gamma \min_{a' \in A} Q\Big(\arg\min_{\mathbf{u}} \Big[r_T(\mathbf{x}_T) + \sum_{t=0:dt:T-dt} \Big(r_t(\mathbf{x}_t) + \frac{1}{2}\mathbf{u}_t^T \mathbf{R} \mathbf{u}_t\Big)\Big]\Big)$$

where we have thus written our Q-update for each iteration. All that is left to do is to write in the environment with our RCS control and allow the agent to interact with this control system. Of course, this large Q term is calculated using the PIController algorithm above and the other terms will all be comprised in the reward function that takes in a $z = (s, a)$ (state, action) pair. It's also extremely relevant to point out that due to the extremely complex high-dimensional relationship between the data it is likely that a very complex DQN (Deep Q network) will be necessary. There are many commonly proposed architectures but one very common one for stabilization is using Q_{target} and Q_{pred} networks were one Q network predicts values and another calculates them according to the bellman equation. The use of two networks here addresses the problem of unstable learning caused by continuously updating the Q-values. This is because in Q-learning, the agent updates the Q-value based on the target:

$$Q_{target} = r + \gamma \max_a Q_{target}(s', a)$$

If this same network were used to calculate and update the target as is down in normal Q-learning, the target would continuously shift as the network learns which makes each update unstable. Using the target network in addition to compute Q_{target} means the target remains more stable during the training (although still not guaranteedly exactly stable due to small oscillations in a discretized action space as opposed to a continuous action space). In essence, there is a decoupling here that is happening almost exactly like the proposed dynamics equation before it is recoupled to encapsulate the true dynamics of the system. From there, the only challenges that remain are problems like hypertuning network parameters, constructing experience replays that allow for good training while also avoiding massive training periods, and turning all the agent inputs into multi-agent systems.

6 Conclusion

In summary, the analysis of the selected paper by Gomez and Vicenc has highlighted significant overlaps between the theoretical foundations of stochastic optimal control and its practical extensions into DMARL. The use of path-integral control as detailed in this study and proven by Kappen [**?**]2005

and rederived by myself, provides a robust framework for addressing the challenges inherent in multi-agent systems, in particular non-linear dynamics. By leveraging the framework they developed, I have provided a clear outline on how DMARL can be effectively applied to extend the results presented in the paper, particularly through reward structuring and policy optimization in high-dimensional state-action spaces. This alignment strengthens the potential for DMARL to autonomously control UAV swarms in dynamic environments, a prospect that directly addresses the scalability and adaptability challenges in autonomous multi-agent planning.

Another key point the paper addresses involves the potential of parallelization to enhance computational efficiency. The authors suggest that their path-integral control methodology can be scaled through parallel sampling, a prospect enabled by the independence of passive dynamics and the use of probabilistic inference. This insight is crucial for DMARL, where training times often explode due to the complexity of both the learning and the dynamics themselves. The authors also pointed out that the parallelization, particularly through the use of GPUs or distributed computing systems, could drastically reduce training times by enabling the simultaneous evaluation of multiple trajectories or policy updates. Furthermore, this approach aligns with the iterative sampling requirements of path-integral control which makes the extension to this process for natural especially with deference to RL.

In conclusion, integrating the principles of stochastic optimal control with the scalability advantages of parallel computing offers a transformative approach to DMARL. This synthesis not only holds promise for efficient training but also enhances the feasibility of deploying real-time, robust, and scalable control systems in multi-agent environments. Future work should explore the practical implementation of these parallelization techniques to validate their impact on reducing computation overhead while maintaining the fidelity of the learned control policies.

References

[1] Gómez, Vicenç, et al. *Real-Time Stochastic Optimal Control for Multi-Agent Quadrotor Systems*. arXiv preprint arXiv:1502.04548, 12 May 2020. https://arxiv.org/abs/1502.04548.

[2] Kappen, H. J. *IOPscience*. Journal of Statistical Mechanics: Theory and Experiment, IOP Publishing, 30 Nov. 2005. https://iopscience.iop.org/article/10.1088/1742-5468/2005/11/P11011.